Cloud Networking

Appunti passo passo

Caching, Amazon ElastiCache e Amazon CloudFront

Documento di partenza: 17 - CN - Caching, ElastiCache and CloudFront.pdf

Obiettivo degli appunti

Capire perche il caching e necessario nelle applicazioni cloud moderne, dove puo essere inserito, come funzionano ElastiCache e CloudFront, quali trade-off introducono TTL e strategie di aggiornamento, e come rispondere alle domande d'esame.

Struttura del documento:

1. Problema: perche le applicazioni moderne hanno bisogno di caching.
2. Concetto base: cache, origin, TTL, cache hit e cache miss.
3. Caching nei diversi layer dell'architettura.
4. Amazon ElastiCache: caching applicativo e query caching.
5. Memcached vs Redis: scelta dell'engine.
6. Strategie di caching: lazy loading e write-through.
7. Amazon CloudFront: CDN, edge location e distribuzioni.
8. Analisi architetturale, criticita e raccomandazioni.
9. Glossario e domande d'esame con risposta immediata.

1. Perche serve il caching nelle applicazioni moderne

Le applicazioni moderne non sono piu sistemi monolitici usati da poche centinaia di utenti. Nei contesti cloud devono servire milioni di utenti distribuiti globalmente, con accessi da browser, dispositivi mobili e dispositivi IoT. Questo aumenta il numero di richieste al secondo e rende la latenza un requisito centrale, soprattutto per applicazioni real-time, e-commerce, gaming, social media e streaming.

Aspetto	Scenario moderno	Problema se non uso cache
Utenti	Milioni di utenti distribuiti globalmente	Il backend e il database ricevono troppe richieste dirette
Accesso	Mobile, web, IoT, dashboard, API	Molti canali generano traffico simultaneo
Requisiti	Risposte quasi immediate e bassa latenza	L'utente percepisce ritardi, timeout o degrado
Dati	Contenuti statici, query, profili, sessioni, feed	Recuperare sempre dall'origine e costoso e lento
Costo	Database e backend scalano con il carico	Si pagano risorse backend anche per richieste ripetitive

Idea chiave

Il caching non serve solo a rendere le risposte piu veloci. Serve anche a ridurre il carico sui servizi downstream: database, origin server, applicazioni EC2, API e storage.

2. Che cos'e il caching

Il caching consiste nel servire una richiesta usando una copia temporanea dei dati invece di interrogare ogni volta la sorgente originale. La copia prende il nome di cache; la sorgente originale prende il nome di origin. La cache e uno strato di storage veloce, temporaneo e vicino al punto in cui la richiesta viene elaborata.

Termine	Significato	Esempio
Cache	Copia temporanea e veloce di un dato	Risultato di una query salvato in RAM
Origin	Sorgente originale del dato	Database, server HTTP, bucket S3, API backend
TTL	Tempo di validita del dato in cache	Profilo utente valido per 300 secondi
Cache hit	Il dato richiesto e presente in cache e non e scaduto	La risposta viene restituita subito dalla cache
Cache miss	Il dato non e presente oppure e scaduto	La richiesta passa all'origin, poi la cache viene aggiornata

```
Flusso logico base del caching

Client -> richiede dato
        -> Cache controlla se il dato e disponibile
            -> HIT: restituisce subito il dato al client
            -> MISS: interroga origin, riceve il dato, aggiorna la cache, risponde al client

Effetto principale:
- latenza piu bassa per cache hit
- meno richieste sull'origin
- migliore scalabilita del backend
```

Due conseguenze sono fondamentali:

- La cache migliora molto le prestazioni quando molti utenti chiedono gli stessi dati.
- La cache introduce il problema della consistenza: il dato copiato puo diventare obsoleto rispetto all'origin.

3. TTL: Time To Live

Il TTL e il tempo, espresso normalmente in secondi, durante il quale una chiave o un oggetto rimane valido in cache. Quando il TTL scade, la richiesta successiva genera un cache miss e il dato deve essere recuperato di nuovo dall'origin.

Scelta del TTL	Vantaggio	Svantaggio
TTL corto	Dati piu freschi e meno rischio di stale data	Piu cache miss, piu traffico verso origin, piu latenza
TTL lungo	Piu cache hit e minore carico su database/origin	Rischio maggiore di restituire dati non aggiornati
TTL uniforme	Configurazione semplice e facile da monitorare	Non tutti i dati hanno la stessa frequenza di aggiornamento
TTL differenziato	Ottimizzazione fine per tipo di dato	Aumenta complessita applicativa e operativa

Regola da esame

Un dato e buon candidato per il caching se e letto spesso, cambia poco, costa molto da calcolare o recuperare, e l'applicazione puo tollerare un certo livello di stale data.

4. Dove si puo implementare la cache

La cache puo essere inserita in diversi layer. Non esiste un unico punto corretto: la scelta dipende dal tipo di dato, dalla latenza da ridurre e dal componente che si vuole alleggerire.

Layer	Tecnica o servizio	Cosa accelera	Esempio
Client	HTTP cache-control headers, browser cache	File statici gia scaricati dal client	CSS, JavaScript, immagini
Web content	Amazon CloudFront	Distribuzione globale di contenuti	Sito statico, video, contenuti dinamici con CDN
Applicazione	Amazon API Gateway cache, ElastiCache	Risposte API o dati usati spesso	Sessioni, token, profili, risultati di chiamate
Database	Amazon ElastiCache	Query lente o ripetitive	Join complessi, leaderboard, feed, dati profilo

```
Architettura logica con cache a piu livelli

Utente
  -> Browser cache
  -> CloudFront edge cache
  -> API / Application cache
  -> ElastiCache
  -> Database / Origin

Obiettivo:
rispondere il piu possibile prima di arrivare al database o all'origin.
```

5. Workload che beneficiano del caching

Tipo di workload	Perche beneficia della cache	Esempi
Read-heavy workloads	Molte letture sugli stessi dati; la cache riduce latenza e IOPS sul database	Social network, gaming, media sharing, portali Q&A
Compute-intensive workloads	Risultati costosi da calcolare possono essere riusati; utile su dataset grandi	Motori di raccomandazione, simulazioni HPC
Contenuti statici globali	Lo stesso contenuto viene richiesto da utenti distribuiti geograficamente	Immagini, CSS, JS, video, documenti
API ad alto traffico	Le risposte ripetitive possono essere servite evitando chiamate al backend	Cataloghi prodotti, news, classifiche

6. Quando conviene usare una cache

Per decidere se mettere un dato in cache bisogna rispondere a quattro domande operative:

10. La query o il calcolo sono lenti o costosi?
11. Il dato e relativamente statico e richiesto spesso?
12. L'applicazione puo tollerare dati temporaneamente non aggiornati?
13. Il beneficio supera complessita, costi e rischi di sicurezza aggiuntivi?

Domanda	Se la risposta e SI	Se la risposta e NO
La query e lenta/costosa?	Caching consigliato: si evita di ripagare il costo ogni volta	La cache potrebbe non dare beneficio sufficiente
Il dato e statico e frequente?	Caching molto efficace	Dato dinamico o raro: rischio cache inutile
Il dato puo essere stale?	Si puo scegliere TTL e strategia adatta	Servono dati real-time: cache rischiosa o TTL minimo
Il beneficio giustifica complessita?	Procedere con monitoraggio e test	Meglio ottimizzare query, schema o backend

Errore frequente

Non bisogna mettere tutto in cache solo perche la cache e veloce. Dati che cambiano continuamente o che vengono letti raramente possono peggiorare il sistema: piu complessita, piu costi, maggiore rischio di inconsistenza.

7. Amazon ElastiCache

Amazon ElastiCache e un servizio AWS completamente gestito per creare data store in-memory key-value. In-memory significa che i dati vengono mantenuti in RAM invece che su disco. Questo permette accesso molto rapido, con latenza sub-millisecondo, ed e utile per dati temporanei, frequentemente richiesti o costosi da ricavare dal database.

Caratteristica	Spiegazione
Fully managed	AWS gestisce provisioning, patching, monitoraggio operativo e aspetti infrastrutturali del servizio
In-memory	I dati sono in RAM: accesso molto piu veloce rispetto a storage su disco
Key-value	Ogni dato viene recuperato tramite una chiave, ad esempio user:123:profile
Scalabile	Supporta scaling in, out, up e down in base al carico
Uso tipico	Query caching, session store, leaderboard, feed, dati di profilo, token temporanei

```
Esempio applicativo con ElastiCache

1. L'applicazione riceve una richiesta per un dato.
2. Controlla ElastiCache.
3. Se il dato e presente e valido: cache hit -> risposta immediata.
4. Se il dato manca o e scaduto: cache miss -> query al database.
5. Il database restituisce il dato.
6. L'applicazione aggiorna ElastiCache.
7. L'applicazione risponde al client.
```

8. Memcached vs Redis in ElastiCache

ElastiCache supporta due engine open-source: Memcached e Redis. Entrambi sono in-memory, ma hanno obiettivi diversi. Memcached e piu semplice e orientato alla scalabilita orizzontale semplice; Redis offre funzionalita piu avanzate, persistenza, replica e failover.

Criterio	Memcached	Redis
Modello	Semplice key-value cache	Key-value avanzato con strutture dati complesse
Scalabilita	Buona scalabilita orizzontale con Auto Discovery	Scalabilita tramite shard e cluster mode
Persistenza	Non supporta persistenza	Supporta persistenza
Alta disponibilita	Non supporta replica nativa nel modello indicato	Replica con primary e read replica, automatic failover
Dati complessi	Modello semplice	Liste, set, sorted set, hash e altri tipi avanzati
Pub/Sub	Non e il caso d'uso principale	Supporta publish/subscribe
Quando sceglierlo	Quando serve il modello piu semplice possibile e bassa manutenzione	Quando servono persistenza, HA, failover o strutture dati avanzate

Scelta rapida

Scegli Memcached per cache semplice e scalabilita orizzontale leggera. Scegli Redis quando vuoi funzionalita avanzate, persistenza, replica, failover automatico o strutture dati piu ricche.

9. Terminologia ElastiCache

Termine	Significato
Node	Unita di RAM sicura e collegata alla rete, con un engine ElastiCache in esecuzione
Cluster	Raggruppamento logico di uno o piu nodi ElastiCache
Endpoint	Indirizzo usato dall'applicazione per connettersi a un nodo o cluster
Memcached cluster	Cluster in cui i dati sono partizionati tra nodi; non duplicati tramite replica
Redis shard / node group	Gruppo di 1-6 nodi correlati; se piu nodi, uno primary e gli altri repliche read-only
Redis cluster / replication group	Raggruppamento logico di uno o piu shard Redis
Cluster mode enabled	Redis puo usare piu shard e partizionare automaticamente i dati per scalabilita

```
Redis: struttura logica semplificata
```

```
Redis cluster / replication group
  -> Shard 1 / node group
       -> Primary node
       -> Replica node 1
       -> Replica node n
  -> Shard 2 / node group
       -> Primary node
       -> Replica node 1
       -> Replica node n

Cluster mode enabled:
- dati partizionati su piu shard
- maggiore scalabilita
- possibile replica all'interno dello shard
```

10. Cache miss penalty

La cache miss penalty e il ritardo aggiuntivo che una richiesta subisce quando il dato non e presente in cache o il TTL e scaduto. Invece di rispondere subito, l'applicazione deve interrogare il database, ricevere il dato e aggiornare la cache.

Evento	Passaggi	Effetto
Cache hit	Controllo cache -> dato valido -> risposta	Latenza molto bassa
Cache miss	Controllo cache -> query database/origin -> aggiornamento cache -> risposta	Latenza maggiore
TTL scaduto	Il dato esiste ma non e piu valido	Si comporta come un cache miss
Preloading	Dati caricati in cache prima delle richieste	Riduce miss iniziali ma aumenta memoria e complessita

Nota progettuale

Se l'applicazione non puo tollerare la cache miss penalty, una possibile soluzione e caricare in anticipo tutti o molti dati nella cache. Questa scelta pero aumenta consumo di memoria e rischio di mantenere dati inutilizzati.

11. Strategie di caching: Lazy loading e Write-through

Le due strategie principali viste nel corso sono lazy loading e write-through. Cambia il momento in cui la cache viene aggiornata: nel lazy loading si aggiorna solo quando serve; nel write-through si aggiorna ogni volta che si scrive nel database.

Criterio	Lazy loading	Write-through
Quando aggiorna la cache?	Solo quando arriva una richiesta e si verifica un cache miss	Ogni volta che il dato viene scritto o aggiornato nel database
Uso tipico	Dati letti spesso e scritti raramente	Dati che devono essere aggiornati in tempo reale
Esempio	Profilo utente, contenuto descrittivo	Leaderboard, news piu popolari, classifiche
Vantaggi	Meno dati inutili in cache; failure di un nodo non e fatale	Cache piu coerente, meno stale data
Svantaggi	Cache miss penalty; possibile stale data	Write penalty; cache churn; dati inutilizzati in cache
Rischio principale	Prima richiesta lenta dopo scadenza o assenza	Ogni scrittura costa di piu perche coinvolge DB e

		cache

```
Lazy loading

Read request -> controlla cache
  -> hit: risponde
  -> miss: legge da database, aggiorna cache, risponde

Write-through

Write request -> aggiorna database
              -> aggiorna cache
              -> risponde

Differenza essenziale:
- Lazy loading ottimizza le letture e aggiorna solo quando serve.
- Write-through privilegia coerenza della cache rispetto al costo delle scritture.
```

12. CloudFront e Content Delivery Network

Il caching con CloudFront riguarda soprattutto la distribuzione globale dei contenuti. Il problema non e solo il carico sul backend, ma anche la distanza geografica tra utente e origin. Piu distanza e piu hop di rete significano maggiore latenza e peggiori prestazioni percepite.

Concetto	Spiegazione
CDN	Sistema globale di server di cache distribuiti geograficamente
POP / edge location	Punto vicino all'utente da cui viene servita la copia del contenuto
Origin	Sorgente da cui CloudFront recupera il contenuto se non e in cache
Contenuti statici	HTML, CSS, JavaScript, immagini, file e video
Contenuti dinamici	Risposte che possono dipendere da posizione, headers, query o logica applicativa
Obiettivo	Ridurre latenza, migliorare velocita di trasferimento e diminuire carico sull'origin

```
Flusso CloudFront

1. L'utente richiede un file o una risorsa.
2. Il DNS instrada la richiesta verso la migliore edge location CloudFront.
3. CloudFront controlla la propria cache.
4. Se il file e in cache: lo restituisce subito all'utente.
5. Se il file non e in cache: lo richiede all'origin.
6. L'origin invia il file alla edge location.
7. CloudFront inoltra il file all'utente e lo salva in cache.
```

13. Amazon CloudFront

Amazon CloudFront e il servizio CDN di AWS. Serve dati, video, applicazioni e API a utenti globali con bassa latenza e alta velocita di trasferimento. E integrato con servizi AWS come Amazon S3 ed EC2, che possono essere usati come origin.

Caratteristica	Impatto architetturale
Prestazioni e scala	Distribuisce contenuti da edge location vicine agli utenti
Sicurezza integrata	Si integra con meccanismi di sicurezza AWS e puo proteggere l'origin
Origin AWS	S3 ed EC2 possono diventare sorgenti del contenuto
Edge functions	Permettono personalizzazione di richieste e risposte vicino all'utente
Lambda@Edge	Esegue codice custom serverless al network edge
CloudFront Functions	Esegue personalizzazioni JavaScript leggere a livello CDN

14. Tipi di distribuzioni CloudFront

Tipo di distribuzione	Quando si usa	Origin possibile
Web content	Per contenuti HTTP/HTTPS, statici o dinamici	HTTP server o bucket S3
Video on demand	Per video che gli utenti possono guardare in qualsiasi momento	HTTP server o bucket S3; video codificati per streaming
Live event	Per streaming di eventi live	AWS Elemental MediaStore o AWS Elemental MediaPackage

Differenza chiave

ElastiCache accelera soprattutto dati applicativi e query vicino al backend. CloudFront accelera la distribuzione globale dei contenuti vicino agli utenti finali.

15. ElastiCache vs CloudFront

Criterio	Amazon ElastiCache	Amazon CloudFront
Livello	Applicazione / database	Edge / distribuzione globale
Tipo di cache	In-memory key-value store	CDN distribuita globalmente
Dati tipici	Query result, sessioni, leaderboard, profili	File statici, web content, video, API, contenuti dinamici cachabili
Riduce carico su	Database e servizi applicativi	Origin server, S3, EC2, backend HTTP
Latenza ridotta	Tra applicazione e dato applicativo	Tra utente finale e contenuto
Problema principale	Cache miss, TTL, coerenza, scelta engine	Invalidation, TTL, cache policy, origin protection
Caso d'uso ideale	Query lente e dati letti spesso	Utenti geograficamente distribuiti

16. Analisi architetturale: Componenti -> Flusso -> Criticita -> Raccomandazioni

Scenario: una piattaforma web globale mostra cataloghi, profili utente, feed e contenuti multimediali. Deve ridurre latenza e carico sul database.

16.1 Componenti

- Client: browser, mobile app, dispositivi IoT o dashboard.
- CloudFront: CDN globale che serve contenuti da edge location.
- Origin: bucket S3, server HTTP o applicazione su EC2.
- Applicazione backend: implementa logica, API e accesso dati.
- ElastiCache: cache in-memory per risultati di query e dati applicativi.
- Database: sorgente autorevole per dati persistenti.

16.2 Analisi del flusso

```
Flusso integrato

Client
  -> CloudFront
      -> cache hit: risposta immediata da edge
      -> cache miss: richiesta all'origin
  -> Backend/API
      -> controlla ElastiCache
          -> cache hit: risposta veloce
          -> cache miss: query database, update cache, risposta
  -> Database
```

16.3 Identificazione criticita

Criticita	Causa	Effetto
Stale data	TTL troppo lungo o invalidazione non gestita	Utente riceve dato vecchio
Cache miss penalty	Dato assente o TTL scaduto	Prima richiesta piu lenta
Cache churn	Dati aggiornati troppo frequentemente	Molte scritture inutili in cache
Costi e complessita	Cache configurata su dati non adatti	Piu infrastruttura senza vantaggio reale
Single point logico	Cache non replicata o engine non adatto	Failure con impatto sulle prestazioni o sul servizio
Policy errate CloudFront	Caching di contenuti personalizzati senza controllo	Rischio privacy o risposta sbagliata

16.4 Raccomandazioni

- Misurare prima i colli di bottiglia in uno scenario simile alla produzione.
- Cacheare prima dati costosi, statici e molto letti.
- Usare TTL differenziati quando le classi di dati hanno frequenze di aggiornamento diverse.
- Usare Redis se servono replica, failover, persistenza o strutture dati avanzate.
- Usare Memcached se serve una cache semplice, leggera e facilmente scalabile.
- Usare CloudFront per utenti distribuiti globalmente e contenuti statici/dinamici cacheable.
- Evitare caching di dati sensibili o personalizzati se non si controllano correttamente cache key, headers e policy.
- Monitorare cache hit ratio, miss ratio, latenza, eviction, saturazione memoria e carico origin.

17. Mini mappa concettuale

```
Caching
  -> serve copie temporanee dei dati
  -> riduce latenza
  -> riduce carico su origin/backend/database
```

```
  -> richiede TTL e gestione stale data

ElastiCache
  -> cache applicativa in-memory
  -> key-value
  -> Memcached: semplice, scalabile, senza persistenza/replica
  -> Redis: avanzato, replica, failover, persistenza, dati complessi

CloudFront
  -> CDN globale
  -> edge location vicine agli utenti
  -> cache di contenuti web, video, API
  -> riduce distanza, hop e carico sull'origin

Decisione progettuale
  -> cosa cacheare?
  -> dove cacheare?
  -> quanto deve durare il dato?
  -> quanto stale data posso tollerare?
```

18. Glossario rapido

Termine	Definizione sintetica
Origin	Sorgente originale del dato: database, server HTTP, S3, EC2 o backend applicativo
TTL	Tempo di validita di un dato in cache prima della scadenza
Cache hit	Dato presente e valido in cache
Cache miss	Dato assente o scaduto; serve interrogare l'origin
Cache miss penalty	Ritardo aggiuntivo dovuto al recupero del dato dall'origin
Stale data	Dato in cache non piu perfettamente aggiornato rispetto all'origin
Lazy loading	Aggiornamento della cache solo quando necessario, tipicamente dopo un miss
Write-through	Aggiornamento della cache insieme alla scrittura sul database
CDN	Rete globale di server cache per consegnare contenuti agli utenti con minore latenza
Edge location / POP	Nodo periferico della CDN vicino all'utente finale
ElastiCache node	Porzione di RAM sicura e collegata alla rete che esegue l'engine scelto
Redis shard	Gruppo di nodi Redis, con primary e possibili repliche
Replication group	Cluster Redis composto da uno o piu shard

19. Domande d'esame con risposta subito sotto

Domanda 1: Che cos'e il caching?

Risposta: Il caching e una tecnica che serve le richieste usando una copia temporanea dei dati invece di interrogare sempre l'origin. Migliora le prestazioni riducendo latenza e carico sui servizi downstream.

Domanda 2: Che differenza c'e tra cache e origin?

Risposta: La cache e la copia temporanea e veloce del dato. L'origin e la sorgente originale e autorevole, come database, server HTTP, bucket S3 o backend applicativo.

Domanda 3: Che cosa indica il TTL?

Risposta: Il TTL indica per quanti secondi un dato in cache resta valido. Quando scade, una richiesta successiva genera un cache miss e il dato deve essere recuperato dall'origin.

Domanda 4: Che cosa sono cache hit e cache miss?

Risposta: Cache hit significa che il dato e presente in cache e non e scaduto. Cache miss significa che il dato non e presente oppure e scaduto, quindi deve essere richiesto all'origin.

Domanda 5: Quali workload beneficiano maggiormente del caching?

Risposta: Soprattutto workload read-heavy, compute-intensive e applicazioni globali con contenuti richiesti frequentemente: social network, gaming, media sharing, Q&A, recommendation engine e HPC.

Domanda 6: Quando conviene mettere un dato in cache?

Risposta: Conviene quando il dato e richiesto spesso, cambia poco, e costoso da calcolare o recuperare, e l'applicazione puo tollerare un certo livello di stale data.

Domanda 7: Quando non conviene usare la cache?

Risposta: Non conviene quando i dati cambiano continuamente, sono letti raramente, devono essere sempre real-time oppure la complessita e il costo della cache superano i benefici.

Domanda 8: Che cos'e Amazon ElastiCache?

Risposta: E un servizio AWS fully managed di caching in-memory key-value, usato per accesso temporaneo e veloce a dati frequentemente richiesti, con latenza sub-millisecondo.

Domanda 9: Perche ElastiCache e piu veloce di un database tradizionale?

Risposta: Perche memorizza i dati in RAM invece che su disco. Inoltre evita di rieseguire query complesse o calcoli costosi quando il risultato e gia disponibile in cache.

Domanda 10: Quali engine supporta ElastiCache?

Risposta: Supporta Memcached e Redis. Memcached e piu semplice e adatto a cache distribuite leggere; Redis offre funzionalita avanzate come persistenza, replica, failover e strutture dati complesse.

Domanda 11: Quando sceglieresti Memcached?

Risposta: Quando serve il modello piu semplice possibile, con bassa manutenzione, multithreading e scalabilita orizzontale tramite Auto Discovery.

Domanda 12: Quando sceglieresti Redis?

Risposta: Quando servono persistenza, alta disponibilita, replica, automatic failover, publish/subscribe o strutture dati avanzate.

Domanda 13: Che cos'e la cache miss penalty?

Risposta: E la latenza aggiuntiva introdotta da un cache miss: l'applicazione deve interrogare il database o l'origin, ricevere il dato, aggiornare la cache e solo poi rispondere.

Domanda 14: Differenza tra lazy loading e write-through?

Risposta: Nel lazy loading la cache viene aggiornata solo quando serve, tipicamente dopo un miss. Nel write-through la cache viene aggiornata ogni volta che il dato viene scritto nel database.

Domanda 15: Quale vantaggio ha il lazy loading?

Risposta: Mantiene in cache meno dati inutili e aggiorna solo cio che viene effettivamente richiesto. E adatto a dati letti spesso ma scritti raramente.

Domanda 16: Quale svantaggio ha il lazy loading?

Risposta: Introduce cache miss penalty e puo restituire stale data se il TTL o la logica di aggiornamento non sono gestiti bene.

Domanda 17: Quale vantaggio ha il write-through?

Risposta: Riduce il rischio di stale data, perche la cache viene aggiornata insieme al database.

Domanda 18: Quale svantaggio ha il write-through?

Risposta: Ogni scrittura costa di piu, perche coinvolge sia database sia cache. Inoltre puo generare cache churn e memorizzare dati che non verranno mai letti.

Domanda 19: Che cos'e una CDN?

Risposta: Una Content Delivery Network e un sistema globale di server cache che distribuisce contenuti da edge location vicine agli utenti, riducendo latenza, distanza e numero di hop.

Domanda 20: Che cos'e Amazon CloudFront?

Risposta: E il servizio CDN di AWS che consegna dati, video, applicazioni e API globalmente con bassa latenza e alta velocita. Si integra con S3, EC2, Lambda@Edge e CloudFront Functions.

Domanda 21: Come funziona CloudFront in caso di cache miss?

Risposta: CloudFront riceve la richiesta, controlla la cache edge, non trova il contenuto, lo richiede all'origin, lo riceve, lo inoltra all'utente e lo memorizza nella edge cache.

Domanda 22: Quali tipi di distribuzione CloudFront sono indicati nel corso?

Risposta: Web content per contenuti HTTP/HTTPS, Video on demand per video guardabili in qualsiasi momento, Live event per eventi in streaming live.

Domanda 23: Differenza principale tra ElastiCache e CloudFront?

Risposta: ElastiCache accelera dati applicativi e accesso al database vicino al backend. CloudFront accelera la distribuzione dei contenuti vicino agli utenti finali tramite edge location globali.

Domanda 24: Qual e il rischio principale del caching?

Risposta: Il rischio principale e servire dati obsoleti o incoerenti. Per questo bisogna progettare TTL, invalidazione, chiavi di cache e strategia di aggiornamento in modo coerente con i requisiti applicativi.

20. Risposte compatte da memorizzare

Tema	Frase pronta da esame
Caching	Serve richieste da una copia temporanea dei dati per ridurre latenza e carico sull'origin.
TTL	Regola la durata della validita del dato in cache: TTL corto riduce stale data ma aumenta miss; TTL lungo aumenta hit ma rischia dati vecchi.
ElastiCache	Cache in-memory key-value fully managed, adatta a dati frequentemente letti e query costose.
Memcached	Semplice, multithreaded, scalabile orizzontalmente, ma senza persistenza/replica.
Redis	Avanzato, con strutture dati complesse, persistenza, replica e automatic failover.
Lazy loading	Aggiorna la cache solo al bisogno: efficiente ma con cache miss penalty e possibile stale data.
Write-through	Aggiorna cache e database a ogni scrittura: piu coerente ma con write penalty.
CloudFront	CDN globale che serve contenuti da edge location, riducendo distanza, hop, latenza e carico sull'origin.

Sintesi finale

In un'architettura cloud ben progettata, ElastiCache e CloudFront non sono alternative equivalenti: lavorano a livelli diversi. ElastiCache ottimizza l'accesso ai dati applicativi e riduce il carico sul database; CloudFront ottimizza la distribuzione globale dei contenuti e riduce la latenza percepita dagli utenti finali.